

Dependency Injection, Configuration & Profiles

Complete beginner-friendly study notes for your Java + Spring Boot roadmap.

TODAY'S GOAL

Understand how Spring creates and connects objects, how your application receives configuration from outside the code, and how dev/test/prod settings can be kept separate.

CREATE Spring Beans	CONNECT Dependencies	CONFIGURE Properties
SEPARATE Profiles	PROTECT Secrets	TEST Configuration

Simple mental model

Your Java classes are the parts of the application. Spring is the manager that creates those parts and connects them. Configuration is the set of values that can change without rewriting your Java code.

You already learned Controller → Service → Repository. Today you learn how Spring creates those objects and connects that chain for you.

Imagine your Book Catalog application has a `BookController`, a `BookService`, and a `BookRepository`. The Controller needs the Service, and the Service needs the Repository.

In plain Java, you could create everything yourself:

```
BookRepository repository = new BookRepository();
BookService service = new BookService(repository);
BookController controller = new BookController(service);
```

This works, but as an application grows, there can be dozens or hundreds of classes. Manually creating and connecting all of them becomes messy.

Spring's job

Spring can create these objects for you and give each object the other objects it needs. You mainly describe what your classes need; Spring handles the wiring.

That automatic object creation and connection is the main idea behind **Dependency Injection**.

Before Dependency Injection, make sure the word **object** is clear.

A class is like a blueprint. An object is an actual thing created from that blueprint.

```
class BookService {  
    void addBook() {  
        System.out.println("Book added");  
    }  
}  
  
BookService service = new BookService(); // object
```

Here, BookService is the class, while service refers to an actual object created from it.

Why this matters today

Spring mostly works with objects. It creates objects, keeps track of them, and gives one object to another when it is needed.

A **Spring Bean** is simply an object that Spring creates and manages for you.

If Spring creates a `BookService` object and manages it, that object is called a Bean.

```
BookService
  ↓
Spring creates an object
  ↓
Spring manages that object
  ↓
That object is a Spring Bean
```

The word *Bean* sounds complicated, but for now think: **Bean = an object managed by Spring.**

The most common way is to put a Spring annotation above the class.

```
@Service
public class BookService {

    public void addBook() {
        // business logic
    }
}
```

Spring sees `@Service` and understands that this class should be managed as part of the application.

Other common annotations are `@Component` and `@Repository`. They all tell Spring, in different roles, that a class should be managed.

These annotations all help Spring find and manage classes. The main difference is the **role the class is supposed to play**.

Annotation	Use it for	Example
@Component	A general Spring-managed class.	EmailSender
@Service	Business/application logic.	BookService
@Repository	Database/data-access logic.	BookRepository

For your Book Catalog project, you will commonly see:

```
@RestController
class BookController { ... }

@Service
class BookService { ... }

@Repository
class BookRepository { ... }
```

Don't memorize blindly

The important idea is not the annotation name. Ask: **what job does this class perform?** Controller handles HTTP, Service handles application logic, Repository handles data access.

A **dependency** is an object that another object needs to do its job.

Example: BookService needs BookRepository. Therefore, the Repository is a dependency of the Service.

```
BookService
  ■
  ■■■ needs → BookRepository
```

Dependency Injection means **the needed object is given to the class instead of the class creating it itself.**

Without injection:

```
class BookService {
    private BookRepository repository =
        new BookRepository();
}
```

With injection:

```
class BookService {

    private final BookRepository repository;

    BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

The key difference

The Service no longer says “I will create my Repository.” It says “I need a Repository.” Spring can then provide one.

The previous example is called **constructor injection** because the dependency enters through the constructor.

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

Spring sees that `BookService` needs a `BookRepository`. If the `Repository` is also a Spring Bean, Spring supplies it when creating the `Service`.

```
Spring starts
↓
Finds BookRepository
↓
Creates BookRepository object
↓
Creates BookService
↓
Passes BookRepository into BookService
↓
BookService is ready
```

The `final` keyword means the field should be assigned once and should not later point to a different `Repository`. That makes the dependency clear and prevents accidental replacement.

Interview answer

Constructor injection is preferred because required dependencies are visible in the constructor, can be stored as `final`, and the class is easier to test without relying on hidden setup.

You may see code like this:

```
@Service
public class BookService {

    @Autowired
    private BookRepository repository;
}
```

This is called **field injection**. Spring puts the dependency directly into the field.

It is shorter, but it hides an important fact: the class cannot clearly show what it needs from its constructor.

Constructor injection	Field injection
Dependencies are visible in constructor.	Dependencies are hidden in fields.
Easy to create the class in a test.	Tests may need Spring/reflection setup.
Works naturally with final fields.	Fields generally cannot be final in this style.
Recommended for required dependencies.	Usually discouraged for normal application dependencies.

Rule for your project

When a class needs another object to function, prefer constructor injection.

@Autowired tells Spring to provide a matching dependency. In modern Spring code, if a class has **one constructor**, you normally do not need to write @Autowired on it.

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

Spring understands that the constructor needs BookRepository and tries to find a Bean of that type.

Simple rule

Don't add @Autowired everywhere just because you have seen it in tutorials. Understand what Spring is injecting first.

Spring Boot normally scans your application's package and the packages below it. This is why package placement matters.

```
com.example.bookapi
    ■■■ BookApiApplication.java
    ■■■ controller/
    ■■■ service/
    ■■■ repository/
    ■■■ dto/
```

If your application starts from the root package `com.example.bookapi`, Spring can normally discover components under that package.

That is how annotations such as `@Service` and `@Repository` get noticed.

Common beginner mistake

Putting a Spring component in an unrelated package can cause “Bean not found” errors because Spring may not scan that package.

Sometimes you want to create a Bean yourself instead of putting @Component/@Service on the class.

For this, Spring provides @Configuration and @Bean.

```
@Configuration
public class AppConfig {

    @Bean
    public EmailSender emailSender() {
        return new EmailSender();
    }
}
```

@Configuration marks a class that contains setup instructions.

@Bean says: “the object returned by this method should be managed by Spring.”

Use @Bean when you need to configure or create an object yourself, especially when the class is from a library and you cannot add @Component to it.

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper();
}
```

Easy distinction

@Service → “Spring, manage this class.”

@Bean → “Spring, manage the object returned by this method.”

You do not need to memorize every lifecycle detail today. Understand the basic sequence.

```
Application starts
↓
Spring finds Bean definitions
↓
Creates Bean objects
↓
Connects dependencies
↓
Bean is ready
↓
Application uses it
↓
Application shuts down
↓
Bean is destroyed
```

A Bean can have setup and cleanup work. Spring provides lifecycle hooks for cases where a class needs to prepare resources or release them.

```
@PostConstruct
public void init() {
    // setup work
}
```

For your day-to-day Spring Boot work, focus mainly on **creation** → **dependency injection** → **use** → **shutdown**.

A Bean's **scope** controls how Spring manages its instances.

The default scope is `singleton`. In simple terms, Spring normally creates one shared Bean instance for the application context.

Application

```
■
  ■■■ BookService ■■■
  ■■■ Another class ■■■■ same BookService Bean
```

There are other scopes, such as `prototype`, but you mainly need to know the default behavior today.

Scope	Simple idea
<code>singleton</code>	One Bean instance per Spring application context; default.
<code>prototype</code>	A new instance is created when requested from Spring.

Practical rule

Most services and repositories in a normal Spring Boot application use the default `singleton` behavior. Don't create mutable shared state inside them without understanding the consequences.

Configuration is information your application needs but that should not be hard-coded into Java classes.

Examples: server port, database URL, page-size limits, external service URLs, feature switches.

```
server:
  port: 8080

book:
  pagination:
    default-size: 20
    max-size: 100

external:
  recommendation-url: https://example.com
```

The idea is simple: **code explains what the application does; configuration tells it how to behave in a particular environment.**

Why separate configuration?

The same application may run on your laptop, in testing, and in production. You don't want to rewrite Java code just because a URL or limit changes.

Spring Boot supports both properties and YAML configuration files. Your roadmap uses `application.yml`.

```
server:
  port: 8080

book:
  pagination:
    default-size: 20
    max-size: 100
```

The same idea in properties format would look like:

```
server.port=8080
book.pagination.default-size=20
book.pagination.max-size=100
```

YAML is often easier to read when you have groups of related settings. Do not mix formats just for the sake of it; follow your project's convention.

One way to read a configuration value is @Value.

```
@Value("${book.pagination.max-size}")  
private int maxPageSize;
```

Spring reads the property and places its value into the field.

This is useful for a small number of values, but if one class needs many related settings, a better approach is often @ConfigurationProperties.

Remember

@Value is fine for a small value. Don't create a class with 15 different @Value fields when those values logically belong together.

Your Day 4 roadmap specifically asks you to learn @ConfigurationProperties. The main benefit is that related settings become a normal Java object.

```
book:
  pagination:
    default-size: 20
    max-size: 100
```

```
@ConfigurationProperties(prefix = "book.pagination")
public class PaginationProperties {

    private int defaultSize;
    private int maxSize;

    // getters and setters
}
```

Now the application can inject PaginationProperties wherever it needs these settings.

```
@Service
public class BookService {

    private final PaginationProperties properties;

    public BookService(PaginationProperties properties) {
        this.properties = properties;
    }
}
```

Why this is nice

The configuration is grouped, strongly represented by a Java class, and easier to understand as the number of settings grows.

Depending on your Spring Boot setup, you can register configuration-properties classes using @ConfigurationPropertiesScan or @EnableConfigurationProperties.

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class BookApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(BookApiApplication.class, args);
    }
}
```

Then Spring can discover classes annotated with @ConfigurationProperties.

Don't get stuck on the annotation

The important flow is: YAML values → Spring reads them → values are placed into a Java configuration object → your Service uses that object.

An environment variable is a value provided by the machine or deployment environment instead of being written directly into your source code.

```
DB_URL=jdbc:postgresql://localhost:5432/books
DB_USERNAME=postgres
DB_PASSWORD=secret
```

Your application can read these values through Spring configuration.

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

The important idea is that the same application can receive different values in different environments.

Example

Your laptop might use a local database. Production might use a managed database. The Java application can remain the same while the environment supplies different connection values.

Not every configuration value is a secret.

Usually not secret	Usually secret
Server port	Database password
Pagination limit	API key
Feature flag	JWT signing secret
Public service URL	Private credentials

A secret is something that could let someone access a system or impersonate your application/user.

Golden rule

Never commit passwords, API keys, private tokens, or production credentials into Git. Put them in environment/deployment secret storage.

The Twelve-Factor App is a set of principles for building applications that are easier to deploy and operate. One important principle for today is: **store configuration in the environment**.

That means the application code should not contain environment-specific values such as production database passwords.

```
BAD
String dbPassword = "myProductionPassword";

GOOD
spring.datasource.password=${DB_PASSWORD}
```

This allows the same application build to move between environments without editing the Java source code.

Simple meaning

Build the application once; provide environment-specific settings when you run it.

A profile lets you activate a particular set of configuration values.

Common profiles are dev, test, and prod.

```
application.yml
application-dev.yml
application-test.yml
application-prod.yml
```

The base file contains common configuration. A profile-specific file can contain values that are different for that environment.

Think of profiles as presets

Dev profile = settings for your laptop.

Test profile = settings for automated/testing environments.

Prod profile = settings for the real deployed application.

```
# application.yml
spring:
  application:
    name: book-api
```

```
book:
  pagination:
    max-size: 100
```

```
# application-dev.yml
server:
  port: 8080
```

```
logging:
  level:
    root: DEBUG
```

```
# application-prod.yml
server:
  port: 8080
```

```
logging:
  level:
    root: INFO
```

Then activate a profile, for example with an environment variable:

```
SPRING_PROFILES_ACTIVE=dev
```

The exact way you activate profiles can vary between local development, IDEs, Docker, and deployment platforms. The important idea is that Spring loads the matching profile configuration.

Good configuration candidates include values that change between environments or should be adjustable without changing Java code.

Good candidate	Why
Port	May differ by deployment setup.
Database URL	Different database per environment.
Pagination max size	Business/application setting may change.
External service URL	May differ between dev and prod.
Log level	Usually more verbose in development.

Avoid scattering configuration values throughout Java classes.

In Day 2 you added pagination. Today, remove hard-coded limits from your business code.

Instead of:

```
int maxPageSize = 100;
```

use configuration:

```
book:
  pagination:
    max-size: 100
```

Then bind it to a Java configuration class and inject that class into the Service.

This gives you a useful real-world lesson: **behavior that may change should not automatically require a code change.**

Suppose your Book Catalog later calls a recommendation service.

Don't do this:

```
String url = "http://localhost:9000/recommendations";
```

Instead:

```
external:  
  recommendation-url: ${RECOMMENDATION_URL}
```

Then inject the configuration into the class that needs it.

Why?

Local development, testing, staging, and production may all use different service addresses. Configuration lets the code stay the same.

Your roadmap asks you to write a small test proving configuration is loaded correctly.

The test should answer a simple question: **when Spring starts with the expected configuration, does my configuration object contain the expected values?**

```
book:
  pagination:
    default-size: 20
    max-size: 100
```

Then your test can verify that the Java properties object has `defaultSize = 20` and `maxSize = 100`.

Testing mindset

Don't only test business calculations. Configuration can also be wrong, and a wrong configuration can break a perfectly correct Service.



This is the big picture you should understand after Day 4.

Say it in your own words

Spring starts the application, finds the classes it should manage, creates their objects, reads configuration, and connects the objects according to their dependencies.

Mistake	Better approach
Using field injection everywhere	Prefer constructor injection.
Hard-coding environment-specific URLs	Put them in configuration/environment variables.
Committing passwords/API keys	Use secret/environment storage.
Putting all configuration into Java constants	Use Spring configuration for adjustable settings.
Creating Beans manually with new everywhere	Let Spring manage application dependencies where appropriate.
Making every class a @Component	Use annotations according to the class's actual role.
Not understanding profiles	Know which environment's configuration is active.

Use your existing **Book Catalog API** from Days 1–3.

Step	What to do
1	Review BookController, BookService and BookRepository. Identify each dependency.
2	Make sure dependencies are injected through constructors.
3	Remove unnecessary field injection.
4	Create a typed PaginationProperties configuration class.
5	Move default page size and maximum page size into application.yml.
6	Configure an external service URL through configuration.
7	Create application-dev.yml and application-prod.yml.
8	Keep secrets out of all committed configuration files.
9	Write a test proving your configuration loads correctly.
10	Run the full test suite and commit the changes.

Do it yourself first

Before looking at a complete solution, try to build the configuration class, wire it into the Service, and create the profiles yourself. The goal is understanding, not copying.

Your API should now behave like a small production-style application.

Requirements:

- Book listing supports a default page size from configuration.
- The application rejects or limits page sizes above the configured maximum.
- The external recommendation URL comes from configuration.
- Dev and prod profiles have different non-secret settings.
- No password/API key is committed to Git.
- A test verifies the configuration values are loaded.
- Controller → Service → Repository still works cleanly.

Example configuration

```
book:
  pagination:
    default-size: 20
    max-size: 100

external:
  recommendation-url: ${RECOMMENDATION_URL}
```

Completion test

Change max-size from 100 to 50 without changing Java code. Restart the application and verify that the application now uses 50. If that works, you have understood the point of external configuration.

- What is a Spring Bean?
- What is Dependency Injection?
- What is the difference between a dependency and a Bean?
- Why is constructor injection preferred?
- Why is field injection discouraged?
- What is the difference between `@Component`, `@Service` and `@Repository`?
- When would you use `@Bean`?
- What does `@Configuration` do?
- What is a Bean scope?
- What is the default Spring Bean scope?
- What is `application.yml`?
- What is the difference between `@Value` and `@ConfigurationProperties`?
- Why use `@ConfigurationProperties`?
- What are environment variables?
- Why should secrets not be committed to Git?
- What is a Spring Profile?
- Why do we use `application-dev.yml` and `application-prod.yml`?
- What does the Twelve-Factor principle about configuration mean?
- How would you make a page-size limit configurable?
- How would you test that configuration was loaded correctly?

Interview trick

Don't answer with only definitions. Give a tiny example. For example: "BookService needs BookRepository, so Spring creates the Repository Bean and passes it into the Service constructor."

- I can explain what a Spring Bean is in simple words.
- I understand what a dependency is.
- I can explain Dependency Injection without memorizing a definition.
- I can write constructor injection myself.
- I understand why field injection is discouraged.
- I know the basic purpose of `@Component`, `@Service` and `@Repository`.
- I know when `@Bean` is useful.
- I understand `application.yml`.
- I can explain why configuration should be outside Java code.
- I can use `@ConfigurationProperties` for grouped settings.
- I understand environment variables and secrets.
- I understand dev/test/prod profiles.
- I created `application-dev.yml` and `application-prod.yml`.
- I tested that my configuration loads correctly.
- I completed the configurable pagination task.
- I can explain today's concepts without looking at the notes.

Day 4 success condition

You should now be able to look at a Spring Boot project and understand not only what the annotations mean, but why Spring is creating the objects, where their settings come from, and how the same application can run with different configuration in different environments.